What Mathematical Work Do AI Tutors Leave for Learners? A Qualitative Study and a Test of Decision-Guided Tutoring

Jingying He, Jiawei Ni, Peilin Ying, Duc Le, and Shayan Ahmad
University of Pennsylvania

Abstract: AI tutors must help students without completing the mathematical reasoning they need to practice. Using TutorMoments-Preview, a dataset of real tutoring conversations, we examine this problem in two ways. First, we code the work left for learners in 38 moments. Explicit instructions about balancing support and challenge produced more replies inviting connections, explanations, or strategies than general tutoring guidance. Second, we design an experiment in which a decision model chooses scaffolding or rigor and a language model writes the tutor's reply. Across 520 moments, we compare two decision systems, Jev and OpenAI Decisions, with the tutor choosing its own approach and with fixed instructions. Automated first-reply pass rates were 68.7% for self-choice, 72.9% with Jev, 73.5% with Decisions, and 74.0% with rigor instructions at every moment. Decision-guided replies had higher pass rates than self-choice, but neither system outperformed fixed rigor guidance.

Introduction

A mathematics tutor must decide when to help and when to leave the next step to the learner. *Scaffolding* makes a task accessible, for example through a hint or representation. *Pushing for rigor* asks the learner to explain a relationship, justify a step, or choose a strategy. Too much help removes reasoning the student needs to practice; too little can leave the student stuck. This assistance dilemma is central to tutoring-system design (Koedinger & Aleven, 2007). Research on teacher questioning likewise emphasizes the mathematical thinking elicited from students (Franke et al., 2009).

We use TutorMoments-Preview, a public collection of tutoring conversations and AI continuations (Zhang et al., 2026), for two connected studies. First, we code the mathematical work remaining after a tutor responds. Second, we test whether separating the choice of teaching approach from the writing of a reply improves a smaller language model's tutoring.

Separating instructional planning from language generation has precedents. StratL uses teaching goals to guide tutor replies (Puech et al., 2025); DICE distinguishes whether help is needed from the model's ability to answer (Pal et al., 2026). Eduardo instead trains smaller tutors using feedback on tutoring exchanges (Macina et al., 2026). We test an additional decision step without retraining. Our questions are: **(1)** How do tutoring instructions change the mathematical work left for learners? **(2)** Does a separate scaffold-or-rigor decision improve tutor replies beyond detailed instructions alone, and what does it add in cost and delay?

What TutorMoments contains and how we use it

TutorMoments-Preview contains 462 de-identified transcripts of remote tutoring with students in grades 2–7. Transcripts include dialogue and descriptions of shared-screen activity. A *moment* is a stopping point where the tutor must decide what to do next. Experienced teachers described important situations, the human tutor's action, and what followed. The dataset authors used language models to map these descriptions to categories and combined annotations by majority vote. Their evaluation set contains 520 moments, balanced between 260 scaffolding and 260 rigor opportunities (Zhang et al., 2026).

The original benchmark asks an AI tutor to continue each conversation with a simulated learner. Its *plain prompt* gives general tutoring guidance. Its *evaluation-aware prompt* explains scaffolding, rigor, and over-scaffolding, and asks the tutor to respond appropriately without revealing the assigned category. Explicit guidance improved performance, but models still tended to give too much help and miss opportunities for challenge. The scoring can credit responses combining scaffolding and rigor. The labels therefore describe instructional opportunities, not mutually exclusive forms of good teaching.

Our qualitative study uses released human and AI continuations. Our technical study generates new tutor replies from the same histories. We examine the first response, not the original multi-turn exchange. Our scores consequently measure a different outcome from the published leaderboard.

Study 1: What work does the tutor leave for the learner?

We held the AI model constant, comparing GPT-5.5 under the plain and evaluation-aware prompts, alongside the observed human continuation. We randomly selected 40 mathematics moments from 226 eligible situations after

excluding students used in earlier pilots: one moment per student and 20 per category. Two were excluded because additional learner activity occurred before the human reply, making it unsuitable for comparison with the AI reply at the selected stopping point. The final sample contained 38 moments, 19 per category, and 114 responses.

The unit was the first *tutor response block*: consecutive tutor utterances ending when the learner next spoke or acted. Following a theory-informed content-analysis approach (Hsieh & Shannon, 2005), we developed two codes. *Work Left* records the most demanding mathematical contribution requested but not already supplied: no new work or repetition (W0), calculation or heavily cued recognition (W1), connection or explanation within a narrowed path (W2), or strategy, argument, or diagnosis with meaningful discretion (W3). *Information Supplied* records how much of the current answer the tutor gives: none (R0), general guidance (R1), a specific step or relationship (R2), or the complete answer (R3). NA means the demand or answer information was unclear. These codes describe the response, not whether its demand suits the learner.

Team members reviewed the coded responses and rationales, discussed interpretations, and agreed on final codes. Coding used the preceding dialogue with access to the full history. Cohen's kappa was not calculated because the team used consensus review rather than independent coding. All 114 texts and evidence excerpts were checked against source records. The main comparison is between the 38 pairs of AI replies; human continuations provide a naturalistic reference.

Qualitative findings

Evaluation-aware replies more often left a connection, explanation, or strategy for the learner: 16/38 received W2 or W3, compared with 2/38 plain replies (Table 1). Plain replies supplied the complete active answer in 21/38 cases, versus 5/38 evaluation-aware replies; one plain reply was unclear on this measure.

Table 1: Mathematical work left in the final coding ($n = 38$ per column).

Work left	Plain	Evaluation-aware	Human reference
W0: No new work or repetition	21	2	20
W1: Calculation or cued recognition	15	20	12
W2: Connection or explanation	2	14	1
W3: Strategy, argument, or diagnosis	0	2	0
NA: Unclear demand	0	0	5

Three cases from the coded responses show what these codes capture. In E01, the task was $25 \times 14 = 25 \times (2 \times \star)$. The plain reply supplied 7 and verified the equality (W0, R3). The evaluation-aware reply asked, "What number times 2 makes 14?" It supplied the relationship but left the calculation (W1, R2). A question alone therefore did not indicate substantial reasoning.

In E06, both replies rewrote $5/7 - 3/7$ as $(5 - 3)/7$. The plain reply asked for $5 - 3$ and the final answer (W1, R2). The evaluation-aware reply additionally asked, "And why do we keep the denominator as 7?" The subtraction rule was supplied, but its mathematical explanation remained for the learner (W2, R2).

In E11, both replies gave 17% as the conversion of 0.17 (R3). The plain reply only asked the learner to select that answer (W0). The evaluation-aware reply also asked why it was not 170% (W2). Giving the numerical answer can therefore coexist with leaving a new explanation to the learner. This is why we coded supplied information and remaining work separately.

Distinguishing a calculation from an explanation requires attention to what the learner still has to supply. These examples illustrate the agreed coding, not measured student learning.

Study 2: Testing a decision-guided tutoring pipeline

Decision models and the motivation for our experiment

A *decision model* is a model used to select an action from a predefined set of options, given some context. Its output is a choice that another part of a system can act on. TypeSafe introduced Jev for fast, structured decisions and probabilities (Almeida, 2026). OpenAI's Decisions interface serves a similar role by requesting choices from a language model (OpenAI, n.d.). These systems can therefore select a teaching approach without composing the tutor's reply.

We designed an experiment to test whether this separation could improve mathematics tutoring. A tutor must both decide what kind of help the learner needs and express that help appropriately. Our proposed *pipeline*, a sequence of connected steps, assigns the first task to a decision model and the second to a language model. The

motivation was to give a smaller tutor explicit direction while using a fast decision step to limit added delay. We tested whether this improved responses beyond giving the tutor detailed instructions directly.

Our proposed pipeline

The pipeline has three steps (Figure 1). First, a decision system reads the conversation so far and chooses *scaffolding* or *rigor*. We tested Jev 1.14.0 and OpenAI Decisions using GPT-6 Luna as alternative decision systems. Second, that choice selects one of two instruction sets we wrote. Scaffolding guidance asks for the smallest useful hint while leaving the key calculation to the learner. Rigor guidance asks for a justification, connection, or independent step without giving the reasoning first. Third, Qwen3-32B, the language model serving as the tutor, receives the selected instructions and the full conversation and writes the reply (Qwen Team, 2025). Both instruction sets require attention to the current task and permit context-sensitive support.

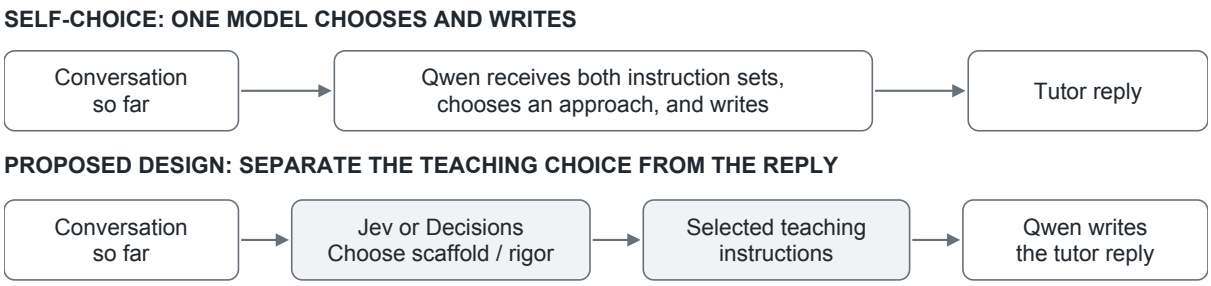


Figure 1: Proposed decision-guided tutoring and our controlled test. The decision step chooses instructions; Qwen writes the reply using the full conversation.

Experimental design and comparison conditions

We evaluated two questions separately: whether a decision system chose the dataset's assigned category, and whether following its choice produced a suitable tutor reply. For the first question, each system returned scaffolding or rigor for all 520 moments. It received the complete preceding conversation, without the assigned label, teacher annotations, or future turns. The selected prompts had been developed on this dataset, so the results describe previously inspected cases rather than an unseen test set.

For the response experiment, Qwen generated three candidate replies at every moment: **self-choice**, receiving both instruction sets and choosing its own approach; **scaffold-guided**, receiving scaffolding instructions; and **rigor-guided**, receiving rigor instructions. All retained the publisher's evaluation-aware prompt and used the same generation settings and response-length limit. This produced 1,560 replies. Each recorded Jev or Decisions choice then selected the corresponding guided reply. For example, a scaffolding decision selected that moment's scaffold-guided candidate. This controlled test reused identical candidates across selectors; it did not run and time a live sequential pipeline.

Self-choice tests whether a separate decision improves on the tutor choosing for itself with equally detailed guidance. Two *fixed-instruction controls* always selected the scaffold-guided candidate or always selected the rigor-guided candidate, regardless of the moment's label. They test whether choosing instructions is useful beyond consistently applying one set. These names describe instructions, not mutually exclusive response behaviors. A final diagnostic selected the candidate matching the dataset label, showing what perfect category selection would achieve with these replies. Neither decision system received that label.

Exact inputs, prompts, and settings are in the supporting materials.

How replies were evaluated

Gemini 2.5 Flash, a separate AI evaluator, rated the three anonymous replies in randomized order, without condition names or dataset labels. It assessed scaffolding, rigor, over-helping, mathematical errors, and fit to the conversation. We then compared those judgments with the assigned category. Passing required the relevant be-

havior, no over-helping or mathematical error, and full context fit. A reply containing both scaffolding and rigor could satisfy either label.

Claude Sonnet 4.6 repeated the ratings for 32 moments, one per student and balanced by category. Comparisons paired replies from the same moment. We estimated uncertainty by repeatedly resampling students, keeping each student's moments together. This avoids treating several moments from one student as independent observations.

Technical findings

Jev's decisions matched the assigned category in 376/520 moments (72.3%); OpenAI Decisions matched it in 386 (74.2%). For reply quality, Jev-guided replies passed in 379 cases and Decisions-guided replies in 382, compared with 357 for self-choice (Table 2). These were gains of 4.2 percentage points for Jev (95% confidence interval: -1.0 to 9.2) and 4.8 points for Decisions (0.2 to 9.5). Always using rigor guidance passed 385 cases, slightly more than either decision system. Jev improved rigor passes from 149 to 180 while reducing scaffolding passes from 208 to 199.

Table 2: Passing first replies. Each row uses the same 520 conversation stopping points.

How instructions are selected	All 520	Scaffold 260	Rigor 260
Qwen chooses its approach	68.7%	80.0%	57.3%
Jev chooses	72.9%	76.5%	69.2%
OpenAI Decisions chooses	73.5%	80.8%	66.2%
Always use scaffold instructions	56.5%	86.2%	26.9%
Always use rigor instructions	74.0%	65.8%	82.3%
Use the dataset label (diagnostic)	84.2%	86.2%	82.3%

The diagnostic uses the assigned label, which an operating tutor would not have. Scores concern our first-reply evaluation.

The evaluator also mattered: on the same 32 cases, Gemini passed 20 self-choice and 19 Jev replies; Claude passed 16 and 22. They agreed on 75/96 reply judgments (78.1%).

Correct category selection did not guarantee a good response: 56 of Jev's 376 category matches still led to failed replies, while 59 of its 144 mismatches passed. In one case, both systems correctly selected scaffolding for a learner working on $1{,}000 - 10$, but Qwen's reply returned to an earlier $338 + 586$ problem.

There was no estimated speed advantage. Qwen's median time to begin answering was 1.20 seconds for self-choice. Adding separately measured decision durations gave estimates of 1.43 seconds with Jev and 1.60 with Decisions. These are reconstructed estimates, not measured live-pipeline times. Estimated model costs per 1,000 replies were \$0.59, \$0.90, and \$1.25, respectively, excluding evaluation.

Discussion and contribution

What the qualitative coding adds

We began with qualitative coding to understand what changes inside a tutor's response when its instructions change. The evaluation-aware prompt left explanation, connection, or strategy work in 16 of 38 replies, compared with two under the plain prompt, while complete-answer giving fell from 21 to five. This suggests that explicit pedagogical guidance can shift the same model from completing the task toward eliciting a mathematical contribution. Yet 20 evaluation-aware replies still left only calculation or cued recognition, and just two left strategy or argument with meaningful discretion. The improvement was mainly toward explaining within a tutor-defined path.

Our two codes make that distinction visible. In E01, asking a question left only a missing-factor calculation; in E06, a supplied procedure still left the denominator explanation open. E11 shows why answer-giving alone is an incomplete measure: the tutor supplied 17% but left the learner to explain why 170% was wrong. For educators and designers, the useful question is what thinking the response still requires. A prompt can request dialogue while supplying nearly all the reasoning. The human reference also frequently left little new work, but these selected moments and first-response codes do not rank human teaching overall. TutorMoments deliberately includes missed opportunities in human tutoring (Zhang et al., 2026).

Did a separate decision model help?

The technical study asked whether a fast decision system could make a smaller tutor more effective without relying on a larger model for every reply. The original benchmark gives a reason to pursue this: evaluation-aware Claude

Opus 4.8 achieved 85.8% appropriate scaffolding and 83.1% appropriate rigor, but the authors reported that it and Gemini 2.5 Pro averaged over ten seconds to complete a response (Zhang et al., 2026). The current project website reports newer models combining strong scores with lower latency and treats cost as a separate concern (Allen Institute for AI, 2026). Affordable, responsive tutoring therefore remains a design objective, rather than a reason to assume all strong models are slow. Our experiment tested decision-guided instructions as one route toward that objective.

The added decision step did not achieve the intended advantage in this experiment. Jev and Decisions improved the pass rate over Qwen choosing its own approach, but both fell below the simpler condition that always supplied rigor instructions. They also added estimated cost and delay. For this configuration, the results favor improving direct instructions before adding a separate chooser. They do not show that every standalone language model is better than every decision model. The original leaderboard establishes that strong tutors exist, but its multi-turn scores cannot be ranked directly against our first-reply scores.

The failures help explain what needs improvement. Matching the assigned category still produced 56 failed Jev-guided replies, including one that answered an earlier problem. Selecting *scaffolding* does not identify the exact misconception or ensure that the tutor follows the current task. Conversely, fixed rigor guidance could still produce support: 202 of its 520 replies combined scaffolding and rigor. This helps explain why it scored well without a separate choice at each moment. The binary decision may be too coarse to add much beyond instructions that already permit both behaviors.

What this makes possible next

Together, the studies provide a way to examine both the response and the system producing it. The codebook asks what mathematical work remains; the experiment asks whether an added component improves that work's delivery. TutorMoments supplies shared conversation histories and teacher-informed targets, so future designs can be compared on the same situations. A next experiment could select a specific action, such as requesting a justification, giving a representation, or leaving an independent step, and use the coding framework to check what reasoning survives in the reply. Smaller models could also be trained on these distinctions. Such variants should be compared with strong direct prompts and timed as complete pipelines on previously unused students or moments.

Our contribution at this stage is the coding framework and a reproducible test showing that this decision-guided design has not yet earned its added complexity. Prompt development used these data, automated evaluators disagreed on some replies, and student learning was not measured. These limits identify the next tests needed before classroom claims. AI tools assisted initial coding, programming, evaluation, and drafting; the team reviewed and agreed on the final qualitative codes.

References

Allen Institute for AI. (2026). *TutorMoments-Preview: When help is unhelpful: Evaluating AI tutors for productive struggle* [Project website and live leaderboard]. Retrieved November 11, 2026, from https://tutormoments.allen.ai/

Almeida, D. (2026, September 15). *Introducing system one models & Jev*. TypeSafe AI. https://typesafe.ai/blog/introducing-system-one-models-and-jev

Franke, M. L., Webb, N. M., Chan, A. G., Ing, M., Freund, D., & Battey, D. (2009). Teacher questioning to elicit students' mathematical thinking in elementary school classrooms. *Journal of Teacher Education*, *60*(4), 380–392. https://doi.org/10.1177/0022487109339906

Hsieh, H.-F., & Shannon, S. E. (2005). Three approaches to qualitative content analysis. *Qualitative Health Research*, *15*(9), 1277–1288. https://doi.org/10.1177/1049732305276687

Koedinger, K. R., & Aleven, V. (2007). Exploring the assistance dilemma in experiments with cognitive tutors. *Educational Psychology Review*, *19*(3), 239–264. https://doi.org/10.1007/s10648-007-9049-0

Macina, J., Kapur, M., & Sachan, M. (2026). *The assistance dilemma: Learning to teach via multi-turn reinforcement learning* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2610.06446

OpenAI. (n.d.). *Decisions* [API documentation]. Retrieved November 11, 2026, from https://developers.openai.com/api/docs/guides/decisions

Pal, S., Ji, K., & Srihari, R. K. (2026). *DICE: Decoupling capability from intervention necessity in LLM tutoring* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2610.04825

Puech, R., Macina, J., Chatain, J., Sachan, M., & Kapur, M. (2025). Towards the pedagogical steering of large language models for tutoring: A case study with modeling productive failure. In W. Che, J. Nabende, E. Shutova, & M. T. Pilehvar (Eds.), *Findings of the association for computational linguistics: ACL 2025* (pp. 26291–26311). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-acl.1348

Qwen Team. (2025). *Qwen3-32B* [Model card]. Hugging Face. Retrieved November 11, 2026, from https://huggingface.co/Qwen/Qwen3-32B

Zhang, A., Ross, A., Patel, K., Bernado, J., Bowie, R., Ribeiro, A. T., Halper, D., Valayaputtur, H., Andreas, J., Loeb, S., Li, L., Lo, K., & Knight, R. (2026). *When help is unhelpful: Evaluating AI tutors for productive struggle* [Working paper]. Allen Institute for AI. https://tutormoments.allen.ai/static/paper/tutormoments-preview.pdf

Supplementary materials

Coding spreadsheet

GitHub repository